

补题所得，按照知识板块分，可能加上其他地方找到的题

代码实现

[乘法逆元] (<https://oi-wiki.org/math/number-theory/inverse/>)

- 快速幂法求逆元

```
inline ll qpow(ll a, ll b, ll mod = MOD) {
    ll res = 1;
    while (b) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}
ll inv(ll a, ll mod) {
    return qpow(a, mod - 2, mod);
}
```

- 乘法原理（独立事件同时发生）
- 加法原理（互斥事件）

A+B problem

- 核心模型**: 题目给出一群概率如何打表？如何转化题目条件
- 题目条件转化 这道题我认为最难的是读懂同时满足的三条条件

最终所有显示器均有灯管被点亮（也就是说显示器的灯管不能全灭）。最终所有显示器显示的结果均为合法数字。第一排的显示器前后拼接形成的四位十进制数记作 A，第二排的显示器前后拼接形成的四位十进制数记作 B 的话，满足： $A+B=C$ 。（两排显示器从左到右依次作为千位、百位、十位、个位进行拼接，可以存在前导 0）。当我们读完题目可以当一个组合满足第三条条件时同时可以满足前两个。题目要求你对“显示器”做算术运算，你只需要关心**这个显示器显示了哪个数字**。只要它显示了数字，它就一定亮了，也一定合法。其他都是出题人故意写的干扰项。以后遇到这种 n 个条件叠加在一起的概率题药仔细想想是不是里面又逻辑包含/递推关系

- 概率转换思路

每根灯管亮起来的分数概率用乘法逆元转换

```
vi pr80(9);
for (int i = 0; i < 7; i++)
{
    pr80[i] = P[i] * inv(100, MOD) % MOD;
}
```

每个数字亮起来的概率，根据乘法原理亮起来的概率是联动的事件所以用乘法

```

vi deng(10);
for (int p = 0; p < 10; p++)
{
    ll cur = 1;
    for (int i = 0; i < 7; i++)
    {
        if (biao[p][i])
            cur = cur * pr80[i] % MOD;
        else
            cur = cur * (1 - pr80[i] + MOD) % MOD;
    }
    deng[p] = cur;
}

```

- **数位DP正解**:可以容纳高精度, 复杂度

$$O(L \cdot 10^2)$$

L 为数字的位数 (本题中 $L = 4$)

```

int biao[10][7] = {
    {1, 1, 1, 0, 1, 1, 1},
    {0, 0, 1, 0, 0, 1, 0},
    {1, 0, 1, 1, 1, 0, 1},
    {1, 0, 1, 1, 0, 1, 1},
    {0, 1, 1, 1, 0, 1, 0},
    {1, 1, 0, 1, 0, 1, 1},
    {1, 1, 0, 1, 1, 1, 1},
    {1, 0, 1, 0, 0, 1, 0},
    {1, 1, 1, 1, 1, 1, 1},
    {1, 1, 1, 1, 0, 1, 1}};

ll qpow(ll a, ll b, ll mod)
{
    ll res = 1;
    a %= mod;
    while (b > 0)
    {
        if (b & 1)
            res = (res * a) % mod;
        a = (a * a) % mod;
        b >>= 1;
    }
    return res;
}

// 求 a 在 mod 下的逆元
ll inv(ll a, ll mod)
{

```

```

    return qpow(a, mod - 2, mod);
}

void solve()
{
    int c;
    cin >> c;
    vi P(8);
    for (int i = 0; i < 7; i++)
    {
        cin >> P[i];
    }
    vi pr80(9);
    for (int i = 0; i < 7; i++)
    {
        pr80[i] = P[i] * inv(100, MOD) % MOD;
    }

    vi deng(10);
    for (int p = 0; p < 10; p++)
    {
        ll cur = 1;
        for (int i = 0; i < 7; i++)
        {
            if (biao[p][i])
                cur = cur * pr80[i] % MOD;
            else
                cur = cur * (1 - pr80[i] + MOD) % MOD;
        }
        deng[p] = cur;
    }

    vi nums;

    // while (c > 0)
    for (int i = 0; i < 4; i++)
    {
        nums.push_back(c % 10);
        c /= 10;
    }

    vector<vi> dp(5, vi(2));
    dp方程设置：第一维是位数，第二维 (0/1) 是是否有进位
    dp[0][0] = 1;
    1. 遍历每一个位置 (个位 -> 十位 -> 百位 -> 千位)
    for (int i = 0; i < 4; i++)
    {
        2. 遍历当前位置接收到的进位 (上一轮传过来的)
        for (int j = 0; j < 2; j++)
        {
            如果当前状态概率为0，说明之前的路走不通，直接跳过 (剪枝)
            if (dp[i][j] == 0)
                continue;
            3. 枚举 A 在这一位可能显示的数字 (0-9)

```

```

        for (int a = 0; a <= 9; a++)
        {
            4. 枚举 B 在这一位可能显示的数字 (0-9)
            for (int b = 0; b <= 9; b++)
            {
                这一位算出来的结果, 必须等于 c 在这一位的数字
                //也就是这个(a + b + j) % 10
                if ((a + b + j) % 10 == nums[i])
                {
                    int nj = (a + b + j) / 10;

                    ll mtf = deng[a] * deng[b] % MOD;
                    dp[i + 1][nj] = (dp[i + 1][nj] + dp[i][j] * mtf) % MOD;
                }
            }
        }
    }

    cout << dp[4][0] << "\n";
}

```

• 小数据AC代码:

```

// Time: 2026-02-03 16:11:12 (周二)
// Author: Keronshans

#include <bits/stdc++.h>
using namespace std;

#define rep(i, a, b) for (int i = (a); i <= (b); ++i)
#define all(x) (x).begin(), (x).end()
#define int long long

using ll = long long;
using ld = long double;
using pii = pair<int, int>;
using pll = pair<ll, ll>;
using vi = vector<int>;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fLL;
const int MOD = 998244353;
const double EPS = 1e-9;

/* ===== */
/*

```

算出每个任务的概率

算出合法数字 x 的概率加起来?
 生成某个数字的概率假设为 p
 所有灯管的 p_i 相乘

比如第一根灯点亮

```

*/
int biao[10][7] = {
    {1, 1, 1, 0, 1, 1, 1},
    {0, 0, 1, 0, 0, 1, 0},
    {1, 0, 1, 1, 1, 0, 1},
    {1, 0, 1, 1, 0, 1, 1},
    {0, 1, 1, 1, 0, 1, 0},
    {1, 1, 0, 1, 0, 1, 1},
    {1, 1, 0, 1, 1, 1, 1},
    {1, 0, 1, 0, 0, 1, 0},
    {1, 1, 1, 1, 1, 1, 1},
    {1, 1, 1, 1, 0, 1, 1}};

ll qpow(ll a, ll b, ll mod)
{
    ll res = 1;
    a %= mod;
    while (b > 0)
    {
        if (b & 1)
            res = (res * a) % mod;
        a = (a * a) % mod;
        b >>= 1;
    }
    return res;
}

// 求 a 在 mod 下的逆元
ll inv(ll a, ll mod)
{
    return qpow(a, mod - 2, mod);
}

void solve()
{
    int c;
    cin >> c;
    vi P(8);
    for (int i = 0; i < 7; i++)
    {
        cin >> P[i];
    }
    vi pr80(9);
    for (int i = 0; i < 7; i++)
    {
        pr80[i] = P[i] * inv(100, MOD) % MOD;
    }
}

```

```
vi deng(10);
for (int p = 0; p < 10; p++)
{
    ll cur = 1;
    for (int i = 0; i < 7; i++)
    {
        if (biao[p][i])
            cur = cur * pr80[i] % MOD;
        else
            cur = cur * (1 - pr80[i] + MOD) % MOD;
    }
    deng[p] = cur;
}

vi nums;
ll ans = 0;
for (int i = 0; i <= c; i++)
{
    int a = i;
    int b = c - i;
    int now = 1;
    for (int j = 0; j < 4; j++)
    {
        now = now * deng[a % 10] % MOD;
        a /= 10;
    }
    for (int j = 0; j < 4; j++)
    {
        now = now * deng[b % 10] % MOD;
        b /= 10;
    }
    ans += now;
    ans %= MOD;
}
cout << ans << '\n';
}

signed main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);

    int T = 1;
    cin >> T; // 多组数据时取消注释

    while (T--)
    {
        solve();
    }

    return 0;
}
```

ADhoc

贪心

Digital Folding

- **核心模型:** 数位贪心显然当答案位数越高越好
- **思维误区 (Bug):**
- **修正逻辑 (Patch):**
- **关键代码:**

```
void solve()
{
    auto gtl = [](int a) -> vi
    {
        vi res;
        while (a > 0)
        {
            res.push_back(a % 10);
            a /= 10;
        }
        reverse(all(res));
        return res;
    };
    int l, r;
    cin >> l >> r;
    if (l == r)
    {
        vi ans = gtl(l);
        reverse(all(ans));
        int a1 = 0;
        for (int i = 0; i < ans.size(); i++)
        {
            a1 *= 10;
            a1 += ans[i];
        }
        cout << a1 << '\n';
        return;
    }
    int k = 0;
    int ri = r;
    while (r > 0)
    {
        k++;
        r /= 10;
    }

    ll hsh = 1;
    int tp = k;
```

```
k--;
while (k--)
{
    hsh *= 10;
}
hsh += 1;
l = max(l, hsh);
if (l > ri)
{
    for (int i = 0; i < tp - 1; i++)
    {
        cout << 9;
    }
    cout << '\n';
    return;
}

vi L = gtl(l);
vi R = gtl(ri);
assert(L.size() == R.size());
vi ans;
bool jiu = 0;
for (int i = 0; i < L.size(); i++)
{
    if (jiu)
    {
        ans.push_back(9);
        continue;
    }
    if (L[i] == R[i])
    {
        ans.push_back(L[i]);
    }
    else
    {
        ans.push_back(R[i] - 1);
        jiu = 1;
    }
}
reverse(all(ans));
reverse(all(R));

int a1 = 0;
for (int i = 0; i < ans.size(); i++)
{
    a1 *= 10;
    a1 += ans[i];
}
int a2 = 0;
for (int i = 0; i < R.size(); i++)
{
    a2 *= 10;
    a2 += R[i];
}
```



```

    cout << max(a1, a2);

    cout << '\n';
}

```

DP

线性划分DP

Blackboard

- **核心模型**: 划分型dp, 注意到它的性质是有单调性的; 判断区间是否能成为一个集合当且仅当 `if ((nums[tp] & mask) == 0)`
- **思维误区 (Bug)**: 注意如果中间塞很多很多0就会tle->可以选择链表来往上找, 记得要加上区间dp和 (我用树状数组处理, 注意下标)
- **修正逻辑 (Patch)**:
- **关键代码**:

```

struct Fenwick
{
    int n;
    vector<long long> tr;

    // 初始化: 传入最大长度 n
    Fenwick(int size) : n(size), tr(size + 1, 0) {}
    // 初始化2: 节省时间开销的init
    void init(int size)
    {
        n = size;
        tr.assign(n + 1, 0);
    }

    // 核心位运算: 获取二进制最低位的 1
    int lowbit(int x)
    {
        return x & -x;
    }

    // 单点修改: 在位置 x 加上 val
    void add(int x, long long val)
    {
        for (; x <= n; x += lowbit(x))
        {
            tr[x] = (tr[x] + val) % MOD;
        }
    }
}

```

```
// 查询前缀和: 查询 [1, x] 的和
long long ask(int x)
{
    long long res = 0;
    for (; x > 0; x -= lowbit(x))
    {
        res = (res + tr[x]) % MOD;
    }
    return res;
}

// 区间查询: 查询 [l, r] 的和
long long range_ask(int l, int r)
{
    if (l > r)
        return 0;
    return (ask(r) % MOD - ask(l - 1) % MOD + MOD) % MOD;
}

};

void solve()
{
    int n;
    cin >> n;
    vi nums(n + 1);
    int lst_idx = 0;
    vi preidx(n + 1);
    rep(i, 1, n)
    {
        cin >> nums[i];
        preidx[i] = lst_idx;
        if (nums[i])
            lst_idx = i;
    }
    // dbg_arr(preidx, 0, preidx.size() - 1);
    Fenwick dp(n + 2);
    dp.add(1, 1);

    for (int i = 1; i <= n; i++)
    {
        int mask = 0;
        int tp = i;
        int cnt = 0;
        int now = 0;

        while (tp > 0 && cnt < 32)
        {
            // cerr<<preidx[tp]<< ' ' << cnt << '\n';

            if ((nums[tp] & mask) == 0)
            {
                mask |= nums[tp];

                tp = preidx[tp];
            }
        }
    }
}
```

```
        now = tp ;
    }
    else
    {
        // now=tp+1;
        break;
    }

    cnt++;
}
// if (tp == 0)
//     now = 1;
dp.add(i + 1, dp.range_ask(now+1, i));
}
cout << dp.range_ask(n + 1, n + 1) << '\n';
}
```
